Mahavir Education Trust's
Shah & Anchor Kutchhi Engineering College
Autonomous Institute Affiliated to University of Mumbai
Bestowed with the University of Mumbai's Best College honor 2024-25.

Name: Mohammed Jaorawala
SYECS1-65

EXPERIMENT NO:7	
Date of Performance:	**15/10/2026**
Date of Submission:	**15/10/2026**

Experiment No:7

DETECT A COMPANY LOGO ON PRODUCT PACKAGING IN A STORE SHELF IMAGE SYSTEMS

7.1Aim: To Detect a company logo on product packaging in a store shelf image systems

7.2 Course Outcome: Analyze visual data using image processing algorithms to recognize objects and their shapes.

7.3 Learning Objectives: Analyze the limitations and challenges associated with Template Matching and explore possible solutions or improvements.

7.4 Requirement: MATLAB

7.5 Related Theory:

Template matching two techniques are there

1. Performance Index Method.
2. Normalized Cross Correlation method.

It is a technique which is used to recognize whether a given part belongs to a particular class of or not. All the parts which belong to a particular class of parts are kept in front of the camera one by one at a time, their images are obtained digitized & stored in a library of parts in memory as templates Ti (k,j) of size (mo x no). Now the class of parts (scene) captured by the camera image is digitized & is given by I(k j) of size (m X n) and stored in the memory.

Now in the memory there are a number of templatesTi (k j) and one image I (k,j). To search whether a given part (template) belongs to the class of parts (image) or not take the

template & scan the gray scale image or digital image I (k , j) from left to right & top to bottom using a raster scanning method.

At each time of translation obtain & index performance is called as performance index Pi(x,y).

Whenever the template matches with the part in the image, the index=0. The match has been filmed and the search is successful. The location of the part is also found. The performance index is given by

$$Pi(x,y)= \sum_{k=1}^{Mo} \sum_{j=1}^{no} I(k=x,j+y)-Ti(k,j)$$

7.6Procedure:

1. set I=1,x=0,y=0,e>0
2. compute Pi(x,y)
3. set y=y+1;if y<=(n-n0) go to step 2
4. set y=0,x=x+1,if x<=(m-m0) go to step2
5. set found =false.

7.7 Program and Output:

Code:

```
%% DETECT A COMPANY LOGO ON PRODUCT PACKAGING IN A STORE SHELF IMAGE
%  Method 1 : Performance Index (sum of absolute differences) - as per procedure
%  Method 2 : Normalized Cross Correlation (NCC)
%
%  Files needed in the same folder (change names below):
%     shelf.jpg  -> store shelf image (scene I)
%     logo.png   -> company logo image (template T)
%  Toolbox needed: Image Processing Toolbox (only for NCC / normxcorr2)

clc; clear; close all;

%% 1. READ IMAGES
sceneFile = 'shelf.jpg';
logoFile  = 'logo.jpg';

I_rgb = imread(sceneFile);
T_rgb = imread(logoFile);

% Convert to gray scale
if size(I_rgb,3) == 3, I = rgb2gray(I_rgb); else, I = I_rgb; end
if size(T_rgb,3) == 3, T = rgb2gray(T_rgb); else, T = T_rgb; end
```

```
% Optional: shrink both images to speed up the raster scan
% (the raster scan with loops is slow on big images)
scale = 0.5;                    % set 1 for no resizing
I = imresize(I, scale);
T = imresize(T, scale);

I = double(I);
T = double(T);

[m,  n ] = size(I);               % scene size   (m x n)
[mo, no] = size(T);               % template size (mo x no)

fprintf('Scene size    : %d x %d\n', m, n);
fprintf('Template size : %d x %d\n', mo, no);

%% ========================================================
%  METHOD 1 : PERFORMANCE INDEX METHOD
%  Pi(x,y) = sum_k sum_j | I(k+x, j+y) - T(k,j) |
%  Perfect match -> Pi = 0
%  ========================================================

% Step 1 : initialise
x = 0;  y = 0;
e = 0.12;                % threshold (tolerance). Real images never give exactly 0
                         % so we accept a match if normalized Pi <= e
found   = false;
bestPi  = inf;
bestXY  = [0 0];
PI_map  = nan(m-mo+1, n-no+1);   % store Pi at every position

tic;
while x <= (m - mo)                    % Step 4 : move down
   y = 0;
   while y <= (n - no)                 % Step 3 : move right
      % Step 2 : compute performance index at (x,y)
      patch = I(x+1 : x+mo, y+1 : y+no);
      Pi = sum(sum(abs(patch - T)));

      % Normalize so that e is independent of image / logo size
      % (0 = perfect match, 1 = worst possible)
      Pi_norm = Pi / (mo * no * 255);

      PI_map(x+1, y+1) = Pi_norm;

      if Pi_norm < bestPi
```

```
            bestPi = Pi_norm;
            bestXY = [x y];
        end
        y = y + 1;
    end
    x = x + 1;
end
t1 = toc;

% Step 5 : decision
if bestPi <= e
    found = true;
end

fprintf('\n--- Performance Index Method ---\n');
fprintf('Minimum Pi (normalized) = %.4f\n', bestPi);
fprintf('Time taken = %.2f s\n', t1);
if found
    fprintf('LOGO FOUND at row = %d, column = %d\n', bestXY(1), bestXY(2));
else
    fprintf('LOGO NOT FOUND (min Pi > threshold e = %.2f)\n', e);
end

%% ========================================================
%  METHOD 2 : NORMALIZED CROSS CORRELATION
%  ========================================================
tic;
C = normxcorr2(T, I);                    % size (m+mo-1) x (n+no-1)
t2 = toc;
[maxC, idx] = max(C(:));
[ypeak, xpeak] = ind2sub(size(C), idx);

% Convert peak position to top-left corner of the matched region
row_ncc = ypeak - mo + 1;
col_ncc = xpeak - no + 1;

nccThreshold = 0.70;                     % NCC close to 1 = good match
fprintf('\n--- Normalized Cross Correlation ---\n');
fprintf('Max correlation = %.4f\n', maxC);
fprintf('Time taken = %.4f s\n', t2);
if maxC >= nccThreshold
    fprintf('LOGO FOUND at row = %d, column = %d\n', row_ncc, col_ncc);
else
    fprintf('LOGO NOT FOUND (max NCC < %.2f)\n', nccThreshold);
end
```

```
%% ======================================================
%  DISPLAY RESULTS
%  ======================================================
Idisp = uint8(I);

figure('Name','Logo Detection','NumberTitle','off');

subplot(2,2,1); imshow(uint8(T)); title('Template (Logo)');

subplot(2,2,2); imshow(Idisp); title('Shelf Image (Scene)');

subplot(2,2,3); imshow(Idisp); hold on;
title('Performance Index Result');
if found
   rectangle('Position', [bestXY(2)+1, bestXY(1)+1, no, mo], ...
           'EdgeColor','g','LineWidth',2);
   text(bestXY(2)+1, max(bestXY(1)-8,5), 'Logo found', ...
       'Color','g','FontWeight','bold');
else
   text(10, 20, 'Logo not found', 'Color','r','FontWeight','bold');
end

subplot(2,2,4); imshow(Idisp); hold on;
title('NCC Result');
if maxC >= nccThreshold
   rectangle('Position', [col_ncc, row_ncc, no, mo], ...
           'EdgeColor','r','LineWidth',2);
   text(col_ncc, max(row_ncc-8,5), 'Logo found', ...
       'Color','r','FontWeight','bold');
else
   text(10, 20, 'Logo not found', 'Color','r','FontWeight','bold');
end

% Performance index map (dark spot = best match)
figure('Name','Performance Index Map','NumberTitle','off');
imagesc(PI_map); colormap(jet); colorbar;
title('Performance Index Pi(x,y)  (minimum = best match)');
xlabel('y (column shift)'); ylabel('x (row shift)');
```

Output:

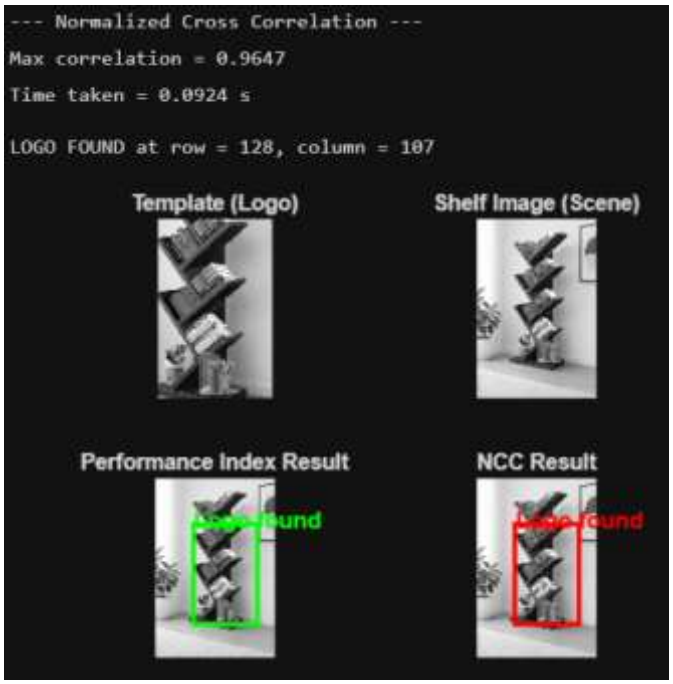
```
Scene size    : 500 x 334
Template size : 280 x 179
```

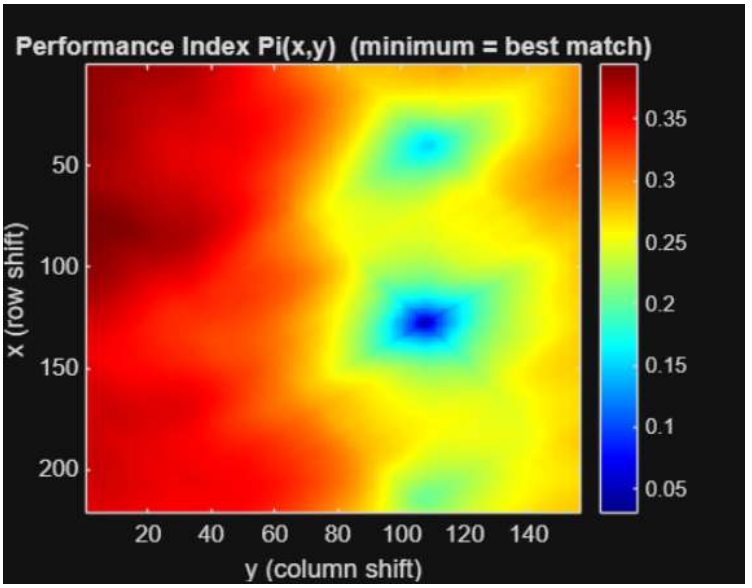
```
--- Performance Index Method ---
Minimum Pi (normalized) = 0.0299
Time taken = 2.36 s

LOGO FOUND at row = 127, column = 106
```

```
--- Normalized Cross Correlation ---
Max correlation = 0.9647
Time taken = 0.0924 s

LOGO FOUND at row = 128, column = 107
```





7.8Conclusion:

Template Matching even though it's a simple technique, one limitation is that the two images being compared should have the same average intensities